

S8: Word Embedding

Full order information

- For some tasks, **order** matters more!
 - Output will also consider order information
 - Translations
 - Voice recognition
- Example
 - Jack loves Ross, Ross doesn't love Jack
 - **v.s** Ross loves Jack, Jack doesn't love ross
 - Are they the same after and BOW representation?

Index-based encoding

- Naive idea: represent the document as it is and **embed each word in one index**, i.e., a number.
- Create a dictionary that maps words to indexes: one word, one **index**
 - {'is': 1, 'awesome': 2, ... 'cool': 891, ... 'unstructured': 1108, ... 'data': 2714...}
- Each document is represented through a sequence of indexes, each index encoding one word
 - Doc: ['Unstructured', 'data', 'is', 'cool'] ~ [1108, 2714, 1, 891]
 - Collection: [[21025, 308, 6, 3, 1050, 207, 8, 2138, 32, 1, 171, 57, 15, 49, 81, 5785, 44, 382, 110, 140, 15,], [5194, 60, 154, 9, 1, 4975, 5852, 475, 71, 5, 260, 12, 21025, 308, 13, 1978, 6, 74, 2395, 5, 613, 73, 6, 5194, 1, 24103, 5,], [1983, 10166, 1, 5786, 1499, 36, 51, 66, 204, 145, 67, 1199, 5194.....]]
- Does it preserve the full order information?

Python: Index encoding

```
c4 = ['AUD is cool', 'word2vec is cool', ..., 'is AUD cool']
tokenized_c4 = [nltk.word_tokenize(doc.lower()) for doc in c4]

# A set for all possible words
words = [j for i in tokenized_c4 for j in i]

## use package
from numpy import array
from sklearn.preprocessing import LabelEncoder
index_encoder = LabelEncoder()
index_encoder = index_encoder.fit(words) # define vocabulary

# encoding
index_encoded2 = [index_encoder.transform(doc) for doc in tokenized_c4]
index_encoded2l = [index_encoder.transform(doc).tolist() for doc in tokenized_c4]
```

Varying in length

- Longer documents have longer representations
- Solution: **padding/truncating**, keep all the doc the same length:

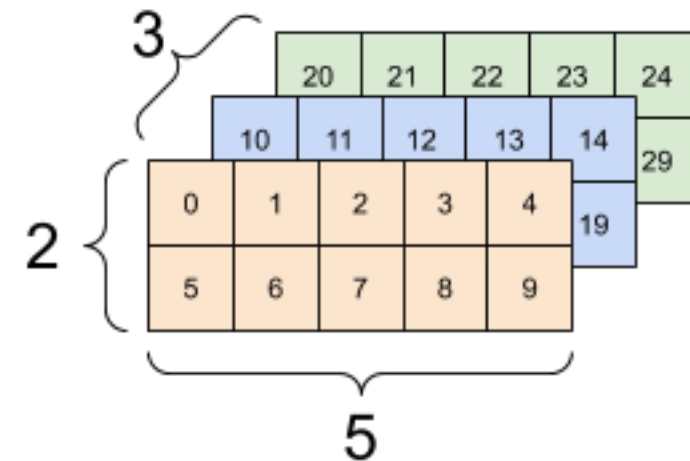
```
[[ 0 0 0 ... 8 215 23]
 [ 0 0 0 ... 29 108 3324]
 [22382 42 46418 ... 483 17 3]
 ...
 [ 0 0 0 ... 59 429 1776]
 [ 0 0 0 ... 55 201 18]
 [ 0 0 0 ... 18 17 1191]]
```

- Question: can you run regression on the vector?
- The index is **NOT** a weight or the value of a unified feature,
 - The vector is **NOT** defined in a unified space
 - The numerical distance between documents that doesn't really exist.
 - 'is' (1) vs. 'awesome' (2) vs. 'cool' (891)

Tensor representation for text

- Advanced neural networks typically use 3d tensors (cubic matrix) as the data input's structure
- Does index encoding create a 3d tensor?

```
[[ 0 0 0 ... 8 215 23]
 [ 0 0 0 ... 29 108 3324]
 [22382 42 46418 ... 483 17 3]
 ...
 [ 0 0 0 ... 59 429 1776]
 [ 0 0 0 ... 55 201 18]
 [ 0 0 0 ... 18 17 1191]]
```



(Ordered) one-hot encoding

- Binary representation of each index
 - 891 – [0,0,0,0,0,1,0,0]
- Each word (index) is further represented by **a binary weight vector** of presence/absence of the words
 - 'Cool' – 891 – [0,0,0,0,0,1,0,0]
 - Is each word defined as a Euclidean space dot?

Hello, I'm Ironman. I have Friday AI	AI	1	0	0	0	0	0	0
	Ironman	0	1	0	0	0	0	0
	Friday	0	0	1	0	0	0	0
	have	0	0	0	1	0	0	0
	Hello	0	0	0	0	1	0	0
	I	0	0	0	0	0	1	0
	I'm	0	0	0	0	0	0	1

- Each document is represented through **a sequence of vectors (dots)**

- ```
[[0,..0,1,0.....,0,0],
 [0,,0,..0,1,0,..0],
 [0,1,0,.....,0],
 [0,..0,1,0,.....,0]]
```

- |        |   |   |   |   |   |   |   |   |
|--------|---|---|---|---|---|---|---|---|
| I      | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| ate    | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
| an     | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| apple  | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| and    | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 |
| played | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| the    | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 |
| piano  | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |

# Python: one-hot encoding

```
c4 = ['AUD is cool', 'word2vec is cool', ..., 'is AUD cool']
tokenized_c4 = [nltk.word_tokenize(doc.lower()) for doc in c4]

words = [j for i in tokenized_c4 for j in i]
words_list = [[i] for i in words]

from sklearn.preprocessing import OneHotEncoder

onehot_encoder = OneHotEncoder(sparse=False)
onehot_encoder = onehot_encoder.fit(words_list)

encoding
onehot_encoded3 = [onehot_encoder.transform([[word] for word in doc])
for doc in tokenized_c4]
```

# Discussions

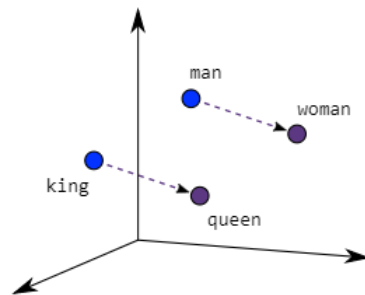
- They perfectly fit with word-level **neural networks frameworks**, e.g., LSTM and RNN
  - Input as **3D** array, process **one word a time**
  - Document as **a sequence of weight vector**
- **No longer** a vector for one document, cannot feed into **classic machine learning frameworks**, e.g., logit regression
- Consists of a possibly very long sequence of 0/1 vectors, leading to a **very large** and **very sparse** representation
  - E.g., a document of 100 words, every words is in a 100 elements long vector, how many 0s?
  - Large and computationally heavy
- Not reflecting the similarity: all words are **equally different**, why?

# Word embedding

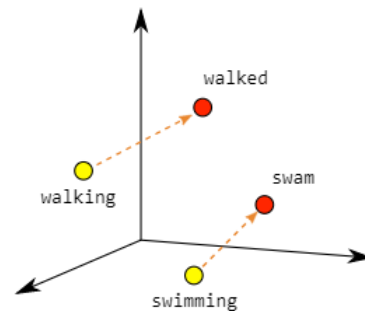
- A family of techniques aiming at **mapping** every **word's semantic meaning** into a **geometric space**
  - Is BOW or one-hot encoding word embedding?
- Idea: map a one-hot vector to a dense, low-dimensional vector that capture semantic relationships between words.
  - 'Good' – 86 –  $[0,0,0,0, \dots, 0,1,0, \dots, 0]$  –  **$[0.21, 0.97, 0.35]$**
- that is
  - in a continuous vector space with a much lower dimension
    - A doc – one hot: a 2D matrix of 1000 (words) by 10000 (elements in vocab) – now: a 2D matrix of 1000 (words) by 100 (real values)
  - the **distance** between any two vectors captures the **semantic relationship** between the **two associated words**

# Semantic meaning

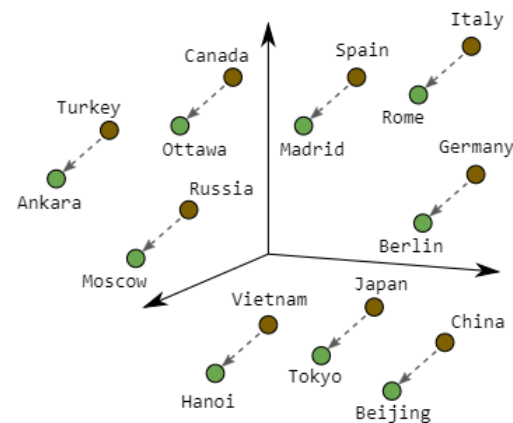
- Each word has its unique vector
- Place vectors of semantically **similar** words **close** to each other
  - Words with similar meaning have similar values in the vector space
  - $\text{vector}(\text{man}) - \text{vector}(\text{women}) + \text{vector}(\text{queen}) \approx \text{vector}(\text{king})$
  - $\text{vector}(\text{Japan}) - \text{vector}(\text{Tokyo}) + \text{vector}(\text{Berlin}) \approx ?$
- Recall: does one-hot has this property?



Male-Female



Verb Tense



Country-Capital

source: <https://developers.google.com/machine-learning/crash-course/embeddings/translating-to-a-lower-dimensional-space>

# Implementation

- How to obtain the value in the word embedding vector?
- A big family of many ML algorithms
  - PCA
  - Word2Vec
    - by Google
    - Distributional hypothesis: **words often have the same neighboring words** tend to be semantically similar and, thus, have **geometrically close** embedding vectors
    - NN (Continuous Bag of Words CBOW and skip-gram)
  - GloVe (Global Vector)
    - by Pennington, et al. at Stanford.
    - Word co-occurrence matrix factorization
  - ....
- In NN models as a hidden layer
  - 'Good' / 86 / [0,0,0,0, .....0,1,0, .....0] – **[beta1,beta2,beta3]**

# Python: Word Embedding

```
from gensim.models import Word2Vec
from matplotlib import pyplot

train model
model = Word2Vec(tokenized_c4, size=2, min_count=1) #size is the size of output
vector

v_is=model.wv['is']
v = model.wv[model.wv.vocab]

words = list(model.wv.vocab)
print(words)
pyplot.scatter(v[:, 0], v[:, 1])

for i, word in enumerate(words):
 pyplot.annotate(word, xy=(v[i, 0], v[i, 1]))
pyplot.show()

sim_is = model.wv.most_similar('is', topn=3)
print(sim_is)
```

# Pre-trained Word2Vec

- Self-trained Word2Vec can be bad if the collection data is **small or not representative**, e.g., a random combination of words
  - E.g., 'king' is closer to 'man' than 'queen'
- Pre-trained Word2Vec
  - Trained on nicely prepared large-scale data on many different contexts
  - A ready-to-use dictionary {word: **keyed vectors**}
- Demo: Word2Vec trained on Google news data (~ 100b words long), containing
  - 3 million unique words and phrases
  - **300-dimensional** keyed vectors.
  - See [code.google.com/archive/p/word2vec/](https://code.google.com/archive/p/word2vec/)
  - 1.53 Gigabytes file, [downloadable](#), **unzip** before use

# Pre-trained GloVe

- Practitioners seem to prefer **GloVe** based on results
- Again, many different data contexts
  - See <https://nlp.stanford.edu/projects/glove/>
- Demo: trained on Wikipedia data with 6b word count
  - a 400,000 words vocabulary
  - an 822 Megabyte zip file with 4 different models (**50**, 100, 200 and 300-dimensional vectors), [downloadable](#) or search [glove.6B.50d.txt](#)
- Question: when better use pre-trained model and when not?

# Python: Pre-trained word embedding

```
from gensim.models import KeyedVectors
filename = 'GoogleNews-vectors-negative300.bin' # make sure the file is unzipped
model = KeyedVectors.load_word2vec_format(filename, binary=True)

sim_is2 = model.wv.most_similar('is', topn=3)
print(sim_is2)

(king - man) + woman = ?
result = model.most_similar(positive=['woman', 'king'], negative=['man'], topn=2)
print(result)

##Pre-trained glove
from gensim.scripts.glove2word2vec import glove2word2vec #convert GloVe format to Word2Vec format
glove_input_file = 'glove.6B.50d.txt' # make sure the file is unzipped
word2vec_output_file = 'glove.6B.50d.txt.word2vec'
glove2word2vec(glove_input_file, word2vec_output_file)

from gensim.models import KeyedVectors
filename = 'glove.6B.50d.txt.word2vec'
model = KeyedVectors.load_word2vec_format(filename, binary=False)

sim_is3 = model.wv.most_similar('is', topn=3)
print(sim_is3)

(king - man) + woman = ?
result = model.most_similar(positive=['woman', 'king'], negative=['man'], topn=2)
print(result)
```

# Mini Group Project 3b

- Use the same reviews for restaurants and films dataset for the previous mini group projects.
- Build a training dataset with the first 400 restaurant reviews and the first 400 movie reviews (ID = [1:400, 501:900]) and use the rest as a test dataset.
- Following step 1, represent each raw document using index-encoding with maximum length of 100 padding.

